Project Two

Ava J. Lindgren

CS 320: Software Test, Automation QA

Professor Karl Lewis

April 19, 2026

Project Two

The unit testing approach that I implemented across the Appointment, Contact, and Task features focused on validating input constraints and verifying correct service behavior. For Appointment, I tested constructor validation as well as service methods including adding, deleting and updating appointments. In testAddAppt(), I verify that a valid appointment was correctly added to the HashMap and that its date and description matched expected values. I also tested invalid cases with empty and null descriptions, using assertThrows to confirm that exceptions were raised. I did similar in the Task and Contact features, and I focused on constructor validation and setter methods by testing both valid and invalid inputs. This aligns fairly closely with the software requirements in which emphasized strict validation rules like the maximum character lengths, non-null fields, correct formatting, and certain features being updatable and non-updatable. Evidence of this alignment can be seen in tests like testPhoneNotTen() in the Contact tests, and testInvalidDate() in the Appointment tests, which directly reflect those constraints.

My JUnit tests' overall quality can be evaluated using the test coverage results. The Task feature achieved approximately 53.8% coverage, the Appointment feature around 50.0% coverage, and the Contact feature about 51.8% coverage. While this indicates that a significant portion of the code was tested, it also shows that there are gaps in coverage

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
milestoneOne	51.8 %	246	229	475
src	51.8 %	246	229	475
main	51.8 %	246	229	475
ContactTest.java	0.0 %	0	133	133
Contact.java	51.9 %	81	75	156
ContactService.java	85.1 %	63	11	74
ContactServiceTest.java	91.1 %	102	10	112

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
moduleFour	53.8 %	284	244	528
src	53.8 %	284	244	528
main	53.8 %	284	244	528
TaskTest.java	0.0 %	0	210	210
Task.java	69.6 %	78	34	112
TaskService.java	100.0 %	59	0	59
TaskServiceTest.java	100.0 %	147	0	147

particularly in some test classes that show 0% coverage (like AppointmentServiceTest and ContactTest in the reports). Despite this, the tests that were executed were effective in validating critical functionality, especially input validation and exception handling. I repeatedly used assertThrows(IllegalArgumentException.class) to try and guarantee that invalid inputs were consistently rejected. Additionally, assertions, like assertEquals() and assertTrue(), confirmed correct behavior after operations of needed tasks like adding and updating objects. Although the coverage is not as high as it could be or I would like for it to be, the tests still did provide meaningful validation of the system's most important requirements, and I have certainly learned a great amount from performing these tests so I can implement better coverage in the future.

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
AvaLindgrenModule5	50.0 %	409	409	818
src	50.0 %	409	409	818
main	50.0 %	409	409	818
AppointmentServiceTest.java	0.0 %	0	300	300
AppointmentService.java	0.0 %	0	59	59
AppointmentTest.java	85.4 %	293	50	343
Appointment.java	100.0 %	116	0	116

Very careful attention was needed when writing the JUnit tests for both technical correctness and efficiency, and it was a struggle much of the time. I did ensure that my code was technically sound by testing both successful and failing scenarios for each feature. In testUpdateApptDate(), I verified that updating an appointment's date correctly changed only the date while preserving the description to make sure that the update logic worked as I intended it to. In the ContactService tests, I validated duplicate prevention by attempting to add the same contact twice and confirming that an exception was thrown- I also checked the efficiency and reliability by isolating tests. This can be seen in the use of @AfterEach methods of AppointmentService.appointments.clear() and TaskService.tasks.clear() which reset shared data structures after each test. This prevents the interference between tests and makes sure of consistent, repeatable results without unnecessary redundancy.

The primary software testing techniques I employed in this project were dynamic testing, unit testing, boundary value testing, and negative testing. Dynamic testing is evident because all tests involved executing the code and comparing expected outputs to actual results. Boundary value testing was used when testing limits such as maximum string lengths (IDs over 10 characters, descriptions over 50 characters, etc.). Negative testing was also a key technique where I intentionally passed invalid inputs such as null values or empty strings to confirm that errors were handled correctly. This is particularly useful in applications that rely on strict validation rules as they help guarantee that invalid data does not enter the system and cause any failures later on.

There were several testing techniques that I did not use in this project, including static testing, integration testing and system testing. Static testing involves analyzing code without executing it, which can be done through formal code reviews or automated analysis tools, which I did not formally apply beyond basic manual review. Integration testing, which focuses on interactions between components, was also not fully utilized since each service was tested independently rather than in combination. System testing, evaluating the complete application, was outside the scope of this project. These techniques are highly important for larger systems. Integration testing would be useful if the Contact, Task, and Appointment services needed to interact with each other, while system testing would ensure that the entire application meets user requirements in a real-world environment.

The mindset I adopted while working on this project emphasized caution and attention to detail. In testing software, it is important to consider how different parts of the code interact, especially since service classes rely on their corresponding model classes for validation. An example of this would be the AppointmentService class depending on the Appointment class's

constructor and setter methods, meaning that invalid data passed into the service would trigger exceptions from the Appointment class. This requires careful test design to make sure that both layers were functioning correctly. I also worked to limit bias by testing not only expected use cases but also edge cases and invalid inputs. Because testing my own code introduces a lot of bias and is quite easy to just assume that the code works as it is intended to, I did choose to intentionally write negative test cases to try and challenge those assumptions. Additionally, it is highly critical to the quality of the software to maintain discipline because cutting corners and taking the quick way out leads to numerous bugs and makes systems harder to maintain. In order to avoid this, I definitely plan to greatly improve my test coverage, as well as including more edge cases and applying additional testing techniques in my future projects to build more reliable and robust software.

References

Boni Garcia. (2017). *Mastering Software Testing with JUnit 5 : Comprehensive Guide to Develop High Quality Java Applications.* Packt Publishing.

Hambling, B., Morgan, P., Samaroo, A., Thompson, G., & Williams, P. (2019). *Software testing : An istqb-bcs certified tester foundation guide - 4th edition.* BCS Learning & Development Limited.